

AI Engineering Simplified

7 Agent Harness Components



Medium

Search

Write



- Home
- Library
- Profile
- Stories
- Stats

- Following
- Python in Plain En...
- Dev Genius
- Generative AI
- Realworld AI Use C...
- The Useful Tech
- Level Up Coding
- AI Mind
- Deep concept
- Top Python Librari...
- The Mac Alchemist
- More

Error Handling

Planning

= Reliable AI Agents

Context Management

Tools

Agent = Model + Harness

Member-only story

7 Agent Harness Components Every AI Developer Needs to Build Reliable AI Agent Systems

The infrastructure layer your agents can't live without in production. Most engineers never build it.



Divy Yadav

Follow

13 min read · Apr 14, 2026



410



4



Your agent just billed a user \$38 on a single query.

Not because it did something complex. Because it summarized the same

document 47 times in a row, it found it had already done the work, then did it again anyway. No crash. No alert. Just a spinning loop and a growing invoice.

You check the model logs. The model was working exactly as trained.

The problem was that everything wrapped around it had no memory of what it already did, no state file, no stop condition. The system had no way to say

“We’ve been here before.”

That is the gap between a demo and a production agent.

In this article, I’ll explain to you the 7 Harnesses you can use to build reliable AI Agents.

The gap nobody warns you about

Building an agent that works once is genuinely easy. Call an LLM, give it tools, let it loop. Twenty lines of Python. You record a demo. It looks clean.

Then you ship it. Real users send unexpected inputs.

A tool call returns empty. Context fills up after forty minutes. Two subagents contradict each other. The model decides to retry something indefinitely.

Everything invisible in the demo becomes a failure in production.

The gap is not model quality. It is harness quality.

“A model without a harness is a brain without a nervous system. The thinking happens. Nothing else does.”

Agent = Model + Harness

This framing changes how you build.

Agent = Model + Harness

Model → reasoning, language, decisions

Harness → everything the model needs to act reliably

If you're not the model, you're the harness.

A harness is every line of code, every config, every execution hook that wraps the model and turns a text generator into something that actually does work.

The model decides what to do. The harness makes sure it can do it safely, repeatedly, at scale.

Most engineers spend 90% of their time on the model: better prompts, newer models, more examples. Production failures almost always live in the 10% they skipped.

The 7 components that actually matter

1. The Control Loop

The loop is the heartbeat of the agent. Without it, you get one model call and one response. That's not an agent, it's a chatbot.

The loop runs the model, reads what it returned, executes any tool calls, feeds the results back in, and repeats until either the model stops calling tools or a step limit fires.

```
while agent_is_running:
    response = call_model(context)

    if response.has_tool_calls:
        results = execute_tools(response.tool_calls)
        append_to_context(results)
        continue

    if response.is_final_answer:
        return response.content

    if step_count > MAX_STEPS:
        return "Task incomplete. Max steps reached."
```

The `MAX_STEPS` line is not optional. It is the difference between a well-behaved agent and the \$38 incident. Build it in before you write a single tool.

A bad loop is worse than no loop. No stop condition, no state tracking, no detection of repeated tool calls means the model can work indefinitely on a task it has already finished.

2. State management

A model is stateless by default. Every API call starts fresh. Without the harness explicitly tracking what happened, the agent has no memory of what it already did, what succeeded, or where it left off.

You need two kinds of state:

Session state covers what happened in this conversation: the conversation history, tool results, current step number.

Persistent state is what survives when the session ends. Progress on a long task. Completed subtasks. Files already processed.

The simplest production state store is a JSON file. Track task progress, already-processed items, and current status. It is readable, debuggable, survives process restarts, and does not require infrastructure.

```
{  
  "task_id": "refactor-auth-module",  
  "completed_files": ["auth.py", "middleware.py"],  
  "pending_files": ["routes.py", "tests/test_auth.py"],  
}
```

```
    "current_step": 3  
  }
```

For a coding agent working across a large codebase, this file is what separates an agent that makes progress from one that re-edits the same file every loop. Git adds versioning on top: agents can track work, roll back mistakes, and branch experiments.

3. Memory

State tracks what the agent did this session. Memory is what it knows across sessions.

Short-term memory is conversation history: every message, tool call, and result is appended to a list passed to the model.

This is cheap to implement and expensive to leave unmanaged. As the list grows, token costs climb, and performance degrades before you hit the hard limit.

Long-term memory is harder. An agent that helps you write code should remember that you prefer explicit error handling over exceptions. One handling customer support should know that a specific customer had a billing issue last week. This typically lives in a vector database for semantic retrieval, or in structured files when the facts are specific.

A good production pattern:

Session **start**:

1. Load AGENTS.md **or** project memory file → inject **into system** prompt
2. Retrieve relevant memories based **on current** task → **add as** context

During session:

3. Maintain rolling conversation history

Session end:

4. Summarize key learnings → write to memory store

The harness handles steps 1, 2, and 4. The model does not manage its own memory. It cannot.

An agent without long-term memory re-learns context on every run. Users notice. They start to feel like the agent is forgetting them even though the model is perfectly capable. That erosion of trust is a harness problem, not a model problem.

4. Tools and the bash escape hatch

Tools are what convert language into action. Without them, the model produces text about doing things. With them, it does them.

Tool design matters more than tool count. Every tool you add costs context (its description lives in the prompt) and increases the chance the model picks the wrong one. Three tools with excellent descriptions will outperform fifteen with vague ones.

A good tool description answers three questions:

- What does this tool actually do?
- When should I use it (not just when I can)?
- What does the output look like so I know it worked?

The bash escape hatch is the architectural move that changes what agents can do. Instead of pre-designing every possible tool, you give the agent access to bash and it writes its own tools on the fly. This is how Claude Code handles open-ended tasks. The model is not constrained to

a fixed tool set. It designs what it needs.

The tradeoff is security, which is why sandbox isolation becomes non-negotiable the moment bash is in play.

Running agent-generated code locally is risky, and a single environment does not scale to concurrent workloads. Sandboxes give agents safe, isolated execution: run code, install packages, inspect files, all without touching your host system. They spin up on demand, fan out across parallel tasks, and tear down when work is done.

A well-configured sandbox also ships with the right defaults: language runtimes, a git CLI, test runners, browsers. This is what lets agents self-verify: write code, run tests, inspect logs, fix failures. The harness builds the environment. The model uses it.

5. Context management

Context rot is one of the sneakiest production failures there is.

The agent was running well for forty minutes. Now it is ignoring its own system prompt. Nothing crashed. No error fired. The context window filled up, the important instructions got buried in the middle, and the model gradually stopped attending to them.

The harness controls what the model sees. The model does not.

Three patterns that actually work in production:

Compaction handles a filling context window by summarizing older conversation history rather than dropping it cold. The key constraint: never compress the original task definition or system prompt. Everything else is negotiable.

Tool output truncation prevents large tool results from flooding context. A 50-page document returned raw from a fetch call will eat your entire budget and crowd out everything useful. The harness keeps the first and last N tokens, stores the full output to the filesystem, and gives the model a pointer if it needs more.

Skills via progressive disclosure solves the startup problem. Loading every tool description at session start bloats context before the agent does anything. Skills load their front-matter on demand, when the model decides it needs that capability. An agent with 50 skills loaded lazily often outperforms one with 10 tools loaded upfront, because the context burden is lower when real work begins.

The production rule: your system prompt and task definition stay visible always. Compress history before you touch those.

6. Planning

A model without planning takes the most obvious next step, whether or not it is part of a coherent path to the goal.

For simple tasks this is fine. For complex multi-step work, it produces incoherence: steps out of order, steps repeated, steps skipped because they were not immediately obvious. The agent can be highly capable and still fail the task because nobody gave it a structure to execute against.

The plan file pattern is the simplest fix that actually works in production:

```
task: Migrate database schema from v1 to v2
steps:
  - Backup current schema          [ ]
  - Generate migration script      [ ]
  - Run migration on staging        [x]
  - Verify data integrity          [ ]
  - Run migration on production    [ ]
  - Update documentation           [ ]
current_step: 4
```

The harness injects this into context at the start of every loop. The agent checks off steps as it completes them. If the session ends, the plan

persists. When the agent resumes, it knows exactly where it is.

Self-verification closes the loop. After completing each step, the agent verifies the result before moving on. The harness can enforce this by running a test suite and feeding back failures. An agent that writes a migration script and immediately verifies it against a staging environment is dramatically more reliable than one that writes and assumes.

The Ralph Loop is worth knowing by name. When an agent finishes its context window on a long task without completing the goal, the Ralph Loop intercepts that exit via a hook, injects the original goal into a fresh context window, and forces continuation. The filesystem makes this possible: each fresh context reads state from the previous iteration. This is how true long-horizon autonomy works across multiple context windows.

7. Error handling

The real world does not cooperate. Tools fail. APIs rate-limit. Files are missing. Models occasionally return output that does not parse.

Without explicit error handling, an agent that hits any of these situations has two bad options: crash, or silently hallucinate around the error as if it did not happen. Both are production failures.

Tool fails:

- Retryable? (timeout, rate limit) → exponential backoff
- Data **error**? → **try** alternative approach
- Permissions **error**? → escalate **to** human

Model output malformed:

- Retry with explicit format reminder
- Three failures → fall back to structured output enforcement

Agent looping:

- Step counter fires → force stop
- Repeated identical tool calls detected → interrupt and redirect

Confidence low:

- Flag for async human review
- Do not block the user while waiting

The escalation path is the most important thing most harnesses do not have. An agent that knows when to stop and ask for help is more useful in production than one that always tries to finish. Build explicit confidence thresholds before you ship.

Every tool call should have a defined failure behavior. Not “handle errors gracefully.” Specific: if this returns empty, do X. If it errors, do Y. If it times out, do Z.

A real trace: what happens inside a production agent

User input: “Summarize the key arguments in last month’s news coverage on EU AI regulation.”

```
Step 1: Plan created
- Search for EU regulation news (last 30 days)
- Read top 5 results
- Extract argument clusters
- Synthesize into structured summary

Step 2: State check
No existing progress. Step counter initialized to 0.

Step 3: Search tool called
Returns 8 articles. Harness truncates each to 500 tokens → adds to context

Step 4: fetch_url() called on top 5 results
Full text stored to filesystem. Agent gets summaries + file pointers.

Step 5: Context check
60% capacity. No compaction needed yet.

Step 6: Synthesis
Agent finds 3 major argument clusters → writes to context.

Step 7: Verification
Agent checks article dates. Two are 45 days old. Flags them.
Re-searches with tighter date filter. Adds 2 new articles.

Step 8: Final output
Structured summary with citations. Step counter: 9. MAX_STEPS: 20.

Step 9: State update
Plan file: all steps complete.
Key findings written to memory store for future sessions.
```

The model wrote the summary. The harness tracked state, managed context, enforced verification, applied the step limit, and wrote to memory. Nine steps, no human intervention, correct output.

That is what a harness actually does. It is not glamorous. It is the difference between a tool that works once and one that works reliably.

The edge cases that will catch you

Hallucination despite tools. The agent has a search tool and uses its training data instead. This happens when the tool description does not make clear when it is required, not just when it is available. Fix: explicit instructions on which questions require a tool call before answering.

Infinite loops. The model retries the same tool with minor variations after getting an empty result. It interprets empty as “try again” rather than “this approach is wrong.” Fix: detect repeated identical tool calls and interrupt with a redirect prompt.

Context overflow. The agent was great for the first 30 minutes. Now it ignores its own instructions. Context filled up slowly, the system prompt got buried, and performance degraded invisibly. Fix: compaction strategy and a hard rule that task definition always appears at context start and end.

Tool misuse. Write before read. Delete instead of archive. These happen when tool descriptions are ambiguous about preconditions. Fix: every description should say when *not* to use the tool, not just when to use it.

Latency explosion. A chain of reasonable tool calls produces a 45-second response. Fix: independent tool calls run concurrently, not sequentially. Measure which harness choices add latency before touching the model.

A thing most engineers don't realize about model-harness coupling

Modern coding agents like Claude Code are post-trained with a model and harness running together. The model learns filesystem operations, bash execution, and planning partly because it was trained while running inside a harness that rewarded these behaviors.

This creates an interesting side effect.

Changing tool logic often degrades model performance, even when the new logic is equivalent. A model trained on a specific patching format will perform worse if you swap the format, even if both formats are logically identical. Training with a harness in the loop creates a kind of overfitting to that harness's design.

The practical consequence: the out-of-the-box harness is not always optimal for your task. On the Terminal Bench 2.0 leaderboard, Opus 4.6 inside Claude Code scores significantly lower than Opus 4.6 inside a custom-tuned harness. Same model, different harness, measurably different ranking.

Most teams have not touched harness optimization at all. That is where real performance is waiting.

When not to use agents

Agents are the wrong tool more often than people want to admit.

Use a deterministic pipeline when the same input always produces the same output through the same steps. Hard-code it. Faster, cheaper, more reliable than anything an agent will do.

Use explicit human gates when a mistake means deleted production data or an email to the wrong person. Separate the agent's recommendation from the execution. Agents make mistakes. Make sure that mistake cannot be irreversible.

Skip agents entirely when the input is structured and the processing is rule-based. An agent adding complexity to a form submission workflow is overengineering, not improvement.

“An agent is not an upgrade from a workflow. It is a different tool for a different class of problem. Know which one you have.”

The clearest signal you are overengineering: every step in your workflow has one correct action, the path is fully defined, and the main reason you want an agent is that agents seem impressive. They do. They are also overkill for deterministic pipelines.

Where to start if you're building from scratch

Add in this order. Each layer solves the most common failure at that stage.

1. **Control loop with a step limit.** Before any tools. `MAX_STEPS = 10` prevents the overnight billing incident before it happens.
2. **State file.** A JSON tracking what happened and what is next. Read it at the start of every loop.
3. **Tool set.** Three to five, well-described. Add more only when you find a specific gap.
4. **Error handling.** Define failure behavior for every tool before shipping.
5. **Context compaction.** Add this when you see degradation in long sessions, not before.
6. **Memory.** Add this when users notice the agent forgetting things it should know.
7. **Planning.** Add this when tasks span multiple sessions or exceed a single context window.

The sequence is not arbitrary. Skip to step 6 without step 2 and you will debug the wrong thing.

Final thought

The best thing about a well-built agent is not what it does when everything works.

It is what it does when something breaks.

As models improve, some of what lives in the harness today will get absorbed natively. Models will get better at planning and self-verification without needing as much prompting support. Some harness complexity will genuinely become unnecessary.

But the engineering around model intelligence, the right tools, durable state, context management, verification loops, these make any model more effective regardless of capability. That is not patching deficiencies. That is system design.

The model gets better every few months. The harness is yours to build.

| *“The model is not your agent. The harness is. Invest accordingly.”*

Every photo is made by the author of this article

Artificial Intelligence

Technology

Programming

Software Development

Data Science



Published in AI Engineering Simplified

Follow

127 followers · Last published 3 days ago

Deep dives into how real AI systems work and how to build systems that work in production. The aim is to explain complex Ai topics in a simplified and practical manner.



Written by Divy Yadav

Follow

1.8K followers · 40 following

I write about how real AI systems work and how to build production-ready, resilient AI applications simply and practically. <https://github.com/dvy246>

